

Grupo:
LUCCAS VANNUCCI
JOÃO PEDRO LANA PINHEIRO
GABRIEL DA SILVA VIEIRA
GUSTAVO LUIS LEITE
GUSTAVO PROVENZANO DE SOUSA

Algoritmos I : Jogo Labirinto

Brasil

2026

Grupo:
LUCCAS VANNUCCI
JOÃO PEDRO LANA PINHEIRO
GABRIEL DA SILVA VIEIRA
GUSTAVO LUIS LEITE
GUSTAVO PROVENZANO DE SOUSA

Algoritmos I : Jogo Labirinto

Algoritmos I : Jogo Labirinto

Centro Universitário Senac
Análise e Desenvolvimento de Sistemas

Orientador: Prof. Dr. Fernando Almeida

Brasil
2026

Resumo

Nosso programa remete a um jogo de labirinto feito por matriz tridimensional, usada especificamente para armazenar três tipos de matrizes bidimensionais diferentes com a mesma quantidade de linhas e colunas. Nossa matriz armazena: mapa, linha e coluna, para justificar o seu tamanho e para ser possível apresentar novos cenários. Cada matriz foi feita manualmente para que o grupo pudesse realizar diferentes combinações de caminhos e paredes, onde o usuário possa se mover e por onde não deva prosseguir.

Palavras-chave: Java; Algoritmos; Modularização; RPG; Desenvolvimento de Software.

Sumário

1	INTRODUÇÃO	5
2	DESENVOLVIMENTO	7
2.1	Introdução e Visão Geral	7
2.2	Funcionalidades Principais do Sistema	7
2.3	Elementos do Cenário e Interface	8
2.3.1	Comandos de Movimentação	8
2.4	Documentação Técnica das Funções	8
2.4.1	Função: main(String[] args)	8
2.4.2	Função: exibirMenu() e exibirRegras()	9
2.4.3	Função: executarJogo(Scanner scanner)	10
2.4.4	Função: copiarMapa(char[] [] mapaOriginal)	11
2.4.5	Função: encontrarJogador(char[] [] labirinto)	12
2.4.6	Função: desenharLabirinto(char[] [] labirinto)	12
2.4.7	Função: jogarPartida(char[] [] labirinto, Scanner scanner)	13
2.5	Fluxograma	13
3	CONCLUSÃO	15

1 Introdução

O código apresentado remete ao contexto das aulas passadas utilizando principalmente matrizes bidimensionais, linhas e colunas, junto de outras funcionalidades como o uso do While e If/else, para que o programa saiba em quais situações algo deve ser feito ou quando deve apresentar um resultado diferente. O programa irá ser apresentado ao terminal, mostrando os caminhos possíveis que o usuário pode percorrer, representados por um espaço vazio, e quais não pode percorrer, representados por uma hashtag “#”, à escolha dos desenvolvedores. O sistema apresenta vários laços de repetição utilizando for, para que seja possível realizar diferentes funções, como o uso para desenhar o tabuleiro, atualizar a posição do jogador, representado pela letra “P”, copiar e reutilizar o mesmo cenário apresentado na tela toda vez que uma nova ação for realizada por “P”, apresentar uma mensagem na tela quando o usuário tentar acessar uma posição na matriz que remete a uma parede, exigindo que o usuário siga o percurso apresentado e uma importação do tipo random, para criar uma função própria para que o sistema pegue um dos mapas apresentados na matriz e decida qual deve ser o próximo cenário. Além disso o sistema conta com uma função do tipo while para que o programa sempre pergunte ao usuário se deseja continuar o jogo, em um mapa diferente, ou não. Se caso o usuário escolher sim, o jogo continua, caso contrário se escolher não, o programa encerra com uma mensagem de agradecimento. Além disso, o nosso código também apresenta as funções, onde cada parte será responsável por uma ação especificada no projeto. O objetivo é deixar o código mais claro e de fácil entendimento, além da organização.

2 Desenvolvimento

O programa realizado em grupo foi feito com cada um pensando em como realizar o jogo de forma eficiente, evitando possíveis erros. Junto de pesquisas e sugestões dadas, chegamos a um resultado onde cada um colaborou para uma parte do nosso projeto. A apresentação das matrizes foi feita em conjunto onde cada um sugeriu um tipo de caminho e quais obstáculos deveriam estar em cada posição. Além disso cada um pegou um pedaço do código para modificar e apresentar possíveis soluções e apresentar novos resultados. Junto a isso, chegamos ao nosso jogo de labirinto, onde conseguimos colocar um sistema onde cada função realiza suas devidas tarefas em tempo real, como a apresentação, os movimentos, o desenho do tabuleiro, a cópia e reuso da mesma matriz apresentada ao terminal, apresentar mensagens de erro caso o usuário siga para direções bloqueadas, mensagens de erro caso o usuário digite opções diferentes das quais o programa aceita, e exibir uma mensagem de continuação, caso desejado.

2.1 Introdução e Visão Geral

O projeto consiste em um ambiente interativo de jogo textual rodando diretamente no terminal do usuário. O grande diferencial técnico da implementação reside na organização lógica do tabuleiro através de matrizes acopladas e no dinamismo proporcionado pela seleção aleatória de cenários.

O software foi projetado seguindo as boas práticas de desenvolvimento orientado a objetos e estruturado, separando a exibição visual da lógica de processamento físico de movimentos e colisões.

2.2 Funcionalidades Principais do Sistema

O sistema é composto por quatro pilares funcionais essenciais:

- **Seleção Aleatória de Mapas:** Utilizando a biblioteca nativa `java.util.Random`, o jogo realiza um sorteio do índice do mapa a cada inicialização, impedindo que as partidas sejam estritamente lineares e aumentando o fator de reatividade.
- **Manipulação Dinâmica de Memória:** Visando proteger as matrizes originais desenhadas manualmente no banco de dados fixo, o sistema faz uma cópia profunda (*deep copy*) dos dados em tempo de execução.

- **Mecânica de Colisão Preditiva:** Antes de atualizar a posição real do caractere do jogador, o algoritmo calcula o destino teórico. Se o destino interceptar uma parede, a ação é barrada sem corromper o estado atual do jogo.
- **Modularização e Limpeza:** Funções específicas isolam as responsabilidades de desenho, busca de coordenadas, clonagem de dados e execução do loop principal.

2.3 Elementos do Cenário e Interface

A matriz textual traduz elementos geométricos e físicos através de caracteres pré-definidos pelos desenvolvedores, conforme apresentado na Tabela 1:

Tabela 1 – Representação dos Elementos na Matriz do Jogo

Caractere	Significado	Comportamento Físico
P	Jogador	Entidade móvel controlada pelo usuário.
#	Parede	Obstáculo intransponível (gera colisão).
[Espaço]	Caminho Livre	Área de livre tráfego para a entidade P.
S	Saída	Objetivo final; ao ser atingido, encerra a partida.

2.3.1 Comandos de Movimentação

O mapeamento de entrada do teclado segue o padrão clássico de jogos de computador (WASD):

- W ou w: Deslocamento para **Cima** (decrementa o índice de linhas).
- S ou s: Deslocamento para **Baixo** (incrementa o índice de linhas).
- A ou a: Deslocamento para a **Esquerda** (decrementa o índice de colunas).
- D ou d: Deslocamento para a **Direita** (incrementa o índice de colunas).

2.4 Documentação Técnica das Funções

Abaixo está detalhada a engenharia lógica por trás de cada método contido no arquivo `Main.java`.

2.4.1 Função: `main(String[] args)`

Ponto de entrada nativo do programa. Gerencia a persistência da aplicação por meio de um laço `while` infinito associado a uma estrutura de seleção múltipla `switch-case`.

Possui tratamento de exceções robusto via bloco `try-catch` para isolar erros do tipo `NumberFormatException` caso o usuário digite texto onde se esperam opções numéricas.

2.4.2 Função: `exibirMenu()` e `exibirRegras()`

Funções utilitárias de saída de dados. Constroem a interface visual do menu e as instruções textuais de jogabilidade no terminal.

```
1 while (executando) {
2     exibirMenu();
3     System.out.println("Escolha uma opcao");
4     String entrada = scanner.next().trim();
5
6     int opcao;
7     try {
8         opcao = Integer.parseInt(entrada);
9     } catch (NumberFormatException e) {
10        System.out.println("Entrada invalida! Digite 1, 2 ou 3.");
11        continue;
12    }
13
14    switch (opcao) {
15        case 1:
16            executarJogo(scanner);
17            break;
18        case 2:
19            exibirRegras();
20            break;
21        case 3:
22            System.out.println("\nObrigado por jogar! Ate a proxima.");
23            executando = false;
24            break;
25        default:
26            System.out.println("Opcao invalida! Escolha 1, 2 ou 3.");
27    }
28 }
29
30 scanner.close();
31 }
```

2.4.3 Função: executarJogo(Scanner scanner)

Atua como o controlador de nível (*Level Controller*). Aloca a matriz tridimensional contendo o banco de labirintos estáticos, realiza o sorteio do mapa ativo e gerencia o loop de continuidade que questiona se o usuário quer jogar novamente em um mapa diferente.

```

1  public static void executarJogo(Scanner scanner) {
2      Random random = new Random();
3      boolean querJogarNovamente = true;
4
5      char[][][] bancoDeMapas = {
6
7          // Mapa 1
8          {
9              {'#', '#', '#', '#', '#', '#', '#'},
10             {'#', 'P', ' ', ' ', '#', ' ', '#'},
11             {'#', '#', '#', ' ', '#', ' ', '#'},
12             {'#', ' ', ' ', ' ', ' ', ' ', '#'},
13             {'#', ' ', '#', '#', '#', '#', '#'},
14             {'#', ' ', ' ', ' ', ' ', ' ', '#'},
15             {'#', '#', '#', '#', '#', ' ', '#'},
16             {'#', ' ', ' ', ' ', ' ', '#', ' ', '#'},
17             {'#', ' ', '#', ' ', '#', ' ', ' ', '#'},
18             {'#', ' ', '#', ' ', ' ', ' ', ' ', '#'},
19             {'#', '#', '#', 'S', '#', '#', '#'}
20         },
21
22         // Mapa 2
23         {
24             {'#', '#', '#', '#', '#', '#', '#'},
25             {'#', 'P', ' ', ' ', ' ', ' ', '#'},
26             {'#', ' ', '#', '#', '#', ' ', '#'},
27             {'#', ' ', '#', ' ', ' ', ' ', '#'},
28             {'#', ' ', '#', ' ', '#', '#', '#'},
29             {'#', ' ', ' ', ' ', ' ', ' ', '#'},
30             {'#', '#', '#', '#', '#', ' ', '#'},
31             {'#', ' ', ' ', ' ', ' ', '#', ' ', '#'},
32             {'#', ' ', '#', ' ', '#', ' ', ' ', '#'},
33             {'#', ' ', '#', ' ', ' ', ' ', ' ', '#'},
34             {'#', '#', '#', '#', 'S', '#', '#'}
35         },
36     }

```

```

37      // Mapa 3
38      {
39          {'#', '#', '#', '#', '#', '#', '#'},
40          {'#', 'P', '#', ' ', ' ', ' ', '#'},
41          {'#', ' ', '#', ' ', '#', ' ', '#'},
42          {'#', ' ', '#', ' ', '#', ' ', '#'},
43          {'#', ' ', ' ', ' ', ' ', '#', ' ', '#'},
44          {'#', '#', '#', '#', '#', ' ', '#'},
45          {'#', ' ', ' ', ' ', ' ', ' ', '#'},
46          {'#', ' ', '#', '#', '#', '#', '#'},
47          {'#', ' ', ' ', ' ', ' ', ' ', '#'},
48          {'#', '#', '#', '#', '#', ' ', '#'},
49          {'#', ' ', 'S', ' ', ' ', ' ', '#'}
50      }
51  };

```

2.4.4 Função: copiarMapa(char[] [] mapaOriginal)

Esta função recebe como parâmetro uma matriz bidimensional de dimensões 11×7 . Através de dois laços de repetição `for` aninhados, aloca um novo espaço em memória e copia elemento por elemento.

$$M_{\text{copia}}[i][j] = M_{\text{original}}[i][j] \quad (2.1)$$

Esse processo garante o isolamento físico dos dados, permitindo reiniciar mapas em seu estado original absoluto.

```

1      public static char[] [] copiarMapa(char[] [] mapaOriginal) {
2
3          char[] [] copia = new char[11][7];
4
5          for (int i = 0; i < mapaOriginal.length; i++) {
6              for (int j = 0; j < mapaOriginal[i].length; j++) {
7                  copia[i][j] = mapaOriginal[i][j];
8              }
9          }
10
11         return copia;
12     }

```

2.4.5 Função: encontrarJogador(char[] [] labirinto)

Varre de forma linear os índices bidimensionais da matriz ativa. Ao localizar a ocorrência do caractere 'P', encapsula os índices de linha e coluna em um vetor unidimensional de inteiros de duas posições.

```
1      public static int[] encontrarJogador(char[] [] labirinto) {
2
3      for (int i = 0; i < labirinto.length; i++) {
4          for (int j = 0; j < labirinto[i].length; j++) {
5
6              if (labirinto[i][j] == 'P') {
7                  return new int[]{i, j};
8              }
9          }
10     }
11
12     return new int[]{1, 1};
13 }
```

2.4.6 Função: desenharLabirinto(char[] [] labirinto)

Imprime delimitadores visuais de interface e percorre toda a estrutura de linhas e colunas adicionando caracteres de espaçamento lateral, garantindo a correta proporção estética do mapa no buffer do console.

```
1      public static void desenharLabirinto(char[] [] labirinto) {
2
3      System.out.println("\n-----");
4
5      for (int i = 0; i < labirinto.length; i++) {
6          for (int j = 0; j < labirinto[i].length; j++) {
7              System.out.print(labirinto[i][j] + " ");
8          }
9          System.out.println();
10     }
11
12     System.out.println("-----");
13 }
```

2.4.7 Função: jogarPartida(char[][] labirinto, Scanner scanner)

Concentra o *Game Loop* interno do labirinto. A cada ciclo renderiza o labirinto atualizado, captura e padroniza a entrada em caixa baixa, calcula preditivamente as novas coordenadas e executa o bloco de validação condicional `if/else` para atualizar a posição do caractere 'P'.

```
1  while (querJogarNovamente) {
2
3      int mapaSorteado = random.nextInt(bancoDeMapas.length);
4
5      char[][] labirinto = copiarMapa(bancoDeMapas[mapaSorteado]);
6
7      System.out.println("\n=== NOVO LABIRINTO GERADO (Mapa Tipo " +
8          (mapaSorteado + 1) + ") ===");
9      System.out.println("Encontre a saida 'S'!");
10
11     jogarPartida(labirinto, scanner);
12
13     System.out.println("\nPARABENS! Voce escapou deste labirinto!");
14
15     System.out.print("\nDeseja jogar novamente em um mapa diferente?
16         (S/N): ");
17     char resposta = scanner.next().toLowerCase().charAt(0);
18
19     if (resposta != 's') {
20         querJogarNovamente = false;
21         System.out.println("\nObrigado por jogar! Ate a proxima.");
22     }
```

2.5 Fluxograma

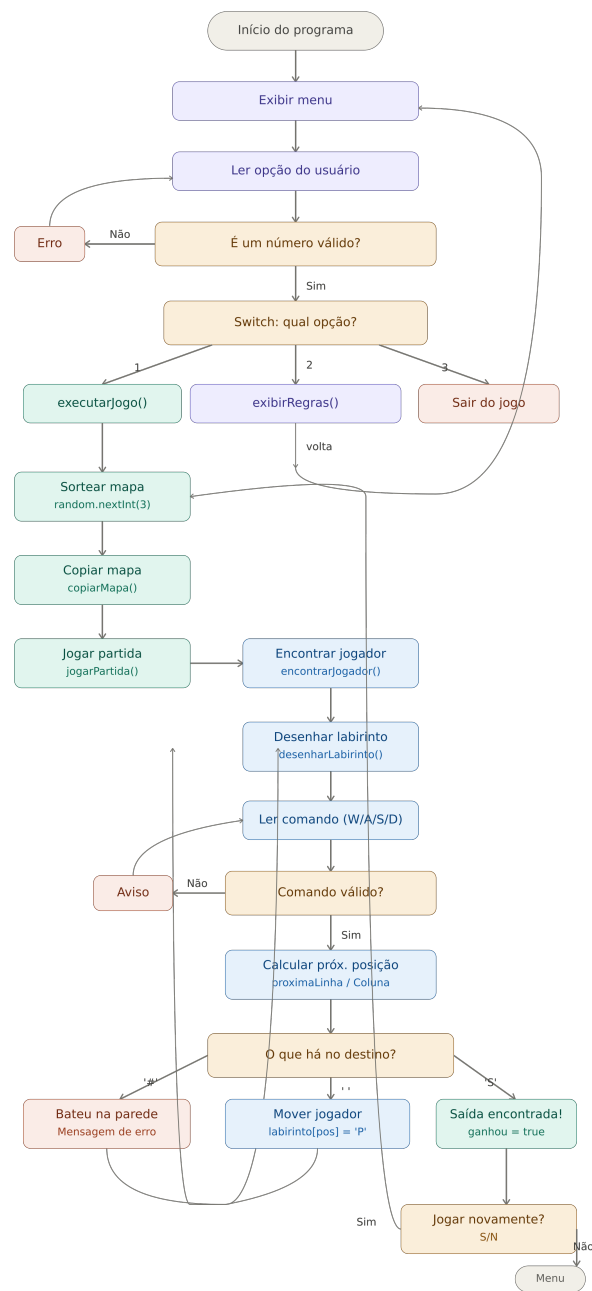


Figura 1 – Escreva aqui a legenda da sua imagem.

3 Conclusão

Portanto, diante a apresentação, todos os integrantes colaboraram pela construções lógica do sistema, onde cada um, com cada conhecimento, ajudou a elaborar o labirinto de forma prática, clara, objetiva, e divertida.